Reconocimiento Computacional de la Ejecución Biomecánica de Ejercicios Mediante una Arquitectura Híbrida CNN-LSTM

Christopher Zúñiga
Escuela de Ciencias de la Computación e Informática
Universidad de Costa Rica
San José, Costa Rica

Adrian Arrieta Orozco
Escuela de Ciencias de la Computación e Informática
Universidad de Costa Rica
San José, Costa Rica

***Resumen*—El análisis computacional del movimiento humano puede modelarse como un problema espacio-temporal: la dimensión espacial codifica la postura corporal en cada instante, mientras que la dimensión temporal captura la dinámica del movimiento a lo largo del tiempo. Este trabajo presenta el diseño de un sistema de reconocimiento automático de la calidad de ejecución de ejercicios de fuerza, clasificando la biomecánica observada como correcta o incorrecta. Para ello, se seleccionaron tres ejercicios —sentadilla (*squat*), dominada (*pull-up*) y peso muerto (*deadlift*)— del conjunto de datos *Workout/Exercises Video Dataset* de Kaggle. La arquitectura propuesta combina una red neuronal convolucional (CNN) para la extracción de características espaciales por fotograma y una red de memoria de corto y largo plazo (LSTM) para modelar la evolución temporal del movimiento a lo largo de secuencias de 30 fotogramas. Como representación de entrada, se emplean los 33 puntos clave del esqueleto humano detectados mediante el modelo BlazePose de MediaPipe, normalizados estructuralmente y enriquecidos con ángulos articulares biomecánicos calculados mediante álgebra vectorial. El enfoque propuesto busca reducir la dependencia de procesamiento de imagen cruda, enfocándose en variables cinemáticas invariantes que faciliten la convergencia del modelo y su generalización en condiciones de grabación variables.**

***Index Terms*—aprendizaje profundo, biomecánica computacional, CNN-LSTM, MediaPipe BlazePose, reconocimiento de actividad humana, reconocimiento de ejercicios**

I. Introducción

El análisis del movimiento humano es una línea de investigación activa en visión por computadora y aprendizaje automático, con aplicaciones en fisioterapia de rehabilitación, entrenamiento deportivo de alto rendimiento, ergonomía laboral y sistemas de asistencia inteligente [1]. Dentro de este dominio, el reconocimiento de la calidad de la ejecución de ejercicios físicos representa un problema con implicaciones directas en la prevención de lesiones biomecánicas.

El análisis del movimiento puede abordarse como un problema de modelado espacio-temporal [1]:

- **Dimensión espacial:** posición y orientación de las articulaciones del cuerpo en cada instante.
- **Dimensión temporal:** dinámica y secuencia del movimiento a lo largo del tiempo.

Los enfoques basados en video ofrecen una representación natural para capturar ambas dimensiones simultáneamente. Sin embargo, el procesamiento directo de píxeles demanda recursos computacionales elevados y puede ser sensible a variaciones de iluminación, ángulo de cámara y características físicas del sujeto.

Una alternativa es estimar la pose del sujeto como un esqueleto de puntos clave (*landmarks*), abstrayendo la información relevante antes de aplicar el modelo de clasificación. Herramientas como MediaPipe BlazePose [2] permiten extraer en tiempo real las coordenadas tridimensionales de 33 articulaciones del cuerpo humano, proporcionando una representación compacta, invariante a cambios de apariencia y apropiada para redes neuronales secuenciales.

Este trabajo propone una arquitectura híbrida CNN-LSTM entrenada sobre secuencias de landmarks y ángulos articulares para clasificar la biomecánica de tres ejercicios de fuerza: *squat*, *pull-up* y *deadlift*. La clasificación binaria (ejecución correcta vs. incorrecta) permite un enfoque simplificado que facilita la obtención de datos etiquetados y la evaluación objetiva del modelo.

El resto del artículo se organiza como sigue: la Sección II presenta los trabajos relacionados; la Sección III describe el conjunto de datos y la formulación del problema; la Sección IV detalla el pipeline de procesamiento; la Sección V especifica la arquitectura propuesta; la Sección VI describe el diseño orientado a objetos del sistema; y la Sección X concluye el trabajo y discute las líneas de trabajo futuro.

II. Trabajos Relacionados

II-A. Redes Neuronales Convolucionales para Datos Visuales

Las redes neuronales convolucionales (CNN) se distinguen del resto de arquitecturas de aprendizaje profundo por su distribución tridimensional de neuronas, lo que las vuelve eficientes para el procesamiento de datos visuales. Su estructura puede describirse mediante tres componentes fundamentales:

1. **Capa convolucional:** procesa regiones localizadas de la entrada mediante filtros que extraen características como bordes, texturas y patrones estructurales.

2. **Capa de agrupación (*pooling*):** reduce las dimensiones espaciales preservando la información esencial, reduciendo la complejidad computacional.
3. **Capa completamente conectada:** clasifica las características extraídas mediante propagación hacia adelante y retropropagación.

II-B. Redes LSTM para Modelado Temporal

Las redes de memoria de corto y largo plazo (LSTM) son un tipo de red neuronal recurrente (RNN) diseñadas para mantener dependencias a largo plazo en secuencias. Cada unidad LSTM incorpora una celda de memoria controlada por tres compuertas [3]:

- **Compuerta de entrada (*input gate*):** decide cuánta información nueva se almacena en memoria.
- **Compuerta de salida (*output gate*):** regula cuánta información se propaga a la capa siguiente.
- **Compuerta de olvido (*forget gate*):** controla qué información previa se descarta de la memoria.

Este mecanismo permite a la LSTM modelar dependencias temporales de largo alcance, como la evolución progresiva de un movimiento a lo largo de múltiples fotogramas.

II-C. Arquitecturas Híbridas CNN-LSTM para Reconocimiento de Actividad

Ullah et al. [3] demostraron que la combinación de características espaciales extraídas por una CNN con el modelado temporal de una LSTM mejora significativamente la precisión en el reconocimiento de acciones en secuencias de video. En esta arquitectura, la CNN actúa como extractor de características por fotograma, y la LSTM modela la secuencia de vectores resultantes para capturar la dinámica del movimiento.

Esta combinación resulta especialmente apropiada para el análisis biomecánico, donde la postura en un instante aislado es insuficiente para evaluar la calidad del movimiento: se requiere analizar cómo las articulaciones se desplazan y coordinan a lo largo del tiempo.

II-D. Representación Esqueletal y Análisis Biomecánico

Ren et al. [1] validan protocolos de preprocesamiento esenciales para el reconocimiento de acción basado en esqueleto, incluyendo la normalización por traslación centrada en la cadera (*hip-center translation*) y el escalado por longitud de torso para lograr invariancia espacial. Winter [4] establece que el análisis del movimiento humano debe fundamentarse en vectores de fuerza y ángulos articulares, lo que justifica la ingeniería de características biomecánicas como etapa de preprocesamiento. Zhou et al. [5] respaldan el uso de representaciones basadas en ángulos y rotaciones para mejorar la eficiencia del aprendizaje en arquitecturas de aprendizaje profundo.

III. Conjunto de Datos y Formulación del Problema

III-A. Conjunto de Datos

El conjunto de datos seleccionado es el *Workout/Exercises Video Dataset* [6], disponible en Kaggle. Este dataset contiene aproximadamente 652 videos distribuidos en 25 categorías de ejercicio. Para el presente trabajo se seleccionaron tres categorías:

Cuadro I
Distribución del subconjunto de datos utilizado

Ejercicio	Videos disponibles	Formato
Deadlift	32	.mp4
Squat	29	.mp4 / .MOV
Pull-up	26	.mp4
Total	**87**	—

III-B. Formulación del Problema

El problema se formula como una clasificación binaria. Dado un video V que contiene la ejecución de uno de los tres ejercicios, el objetivo del modelo es determinar si la biomecánica del movimiento observado corresponde a una ejecución correcta o incorrecta:

$$\hat{y} = f(X_{1:T}), \quad \hat{y} \in \{0, 1\} \tag{1}$$

donde $X_{1:T}$ representa la secuencia de características extraídas sobre $T = 30$ fotogramas del video, y $\hat{y} = 1$ corresponde a ejecución correcta.

La reducción a tres ejercicios específicos y la formulación binaria del problema persiguen aumentar la precisión del modelo al enfocarse en los patrones biomecánicos particulares de cada movimiento, en lugar de abordar un problema de clasificación multi-ejercicio de mayor complejidad.

IV. Metodología de Procesamiento de Datos

IV-A. Extracción de Fotogramas y Detección de Landmarks

Para cada video del conjunto de datos, se extrae una secuencia fija de $T = 30$ fotogramas mediante muestreo uniforme a lo largo de la duración del video. Sobre cada fotograma se aplica el modelo BlazePose de MediaPipe [2], el cual detecta 33 puntos clave (*landmarks*) del cuerpo humano con sus coordenadas tridimensionales (x, y, z).

IV-B. Normalización Estructural

Para que el modelo sea invariante a la distancia de la cámara y a la constitución física del sujeto, se aplica una normalización estructural en dos pasos [1]:

1. **Traslación:** los landmarks se traducen para que la cadera (*hip-center*) quede en el origen del sistema de coordenadas.
2. **Escalado:** las coordenadas se normalizan por la longitud del torso del sujeto, eliminando la dependencia de la escala corporal.

IV-C. Ingeniería de Características Biomecánicas

Las coordenadas de los landmarks se complementan con ángulos articulares relevantes para cada ejercicio. El ángulo θ entre dos segmentos óseos representados como vectores $\mathbf{A}$ y $\mathbf{B}$ se calcula mediante el producto punto de sus vectores unitarios [4], [5]:

$$\theta = \arccos\left(\frac{\mathbf{A}\cdot\mathbf{B}}{\|\mathbf{A}\|\,\|\mathbf{B}\|}\right) \tag{2}$$

Complementariamente, se emplea la ley del coseno para derivar ángulos internos en triángulos formados por tres articulaciones. Los ángulos articulares calculados por ejercicio incluyen:

- **Squat:** ángulo de rodilla y de cadera.
- **Pull-up:** ángulo de codo y de hombro.
- **Deadlift:** ángulo de cadera, rodilla y columna lumbar (estimada).

Este enfoque de ingeniería de características reduce la dimensionalidad de los datos de entrada, elimina redundancias espaciales y permite que la red neuronal converja sobre variables biomecánicas invariantes [5].

IV-D. Exploración Inicial de Datos

Se realizó un análisis exploratorio sobre el dataset de landmarks con el objetivo de verificar que las características extraídas permiten discriminar entre ejecución correcta e incorrecta antes de entrenar el modelo.

IV-D1. Distribución de coordenadas clave: Para cada ejercicio se seleccionó el landmark cuya coordenada Y (posición vertical en imagen, donde $Y = 0$ es la parte superior e $Y = 1$ la inferior) resulta más informativa biomecánicamente: cadera izquierda (landmark 23) para *deadlift*, hombro izquierdo (landmark 11) para *pull-up*, y rodilla izquierda (landmark 25) para *squat*.

La Tabla II resume las estadísticas descriptivas de dichos landmarks por clase. El *squat* muestra la mayor separación: la rodilla correcta tiene una mediana de $Y = 0{,}732$ frente a $Y = 0{,}578$ en la incorrecta, reflejando que la sentadilla correcta alcanza mayor profundidad. El *pull-up* correcto presenta mayor desviación estándar ($\sigma = 0{,}176$ vs $\sigma = 0{,}100$), lo que indica que recorre un rango de movimiento más amplio. El *deadlift* muestra la menor diferencia entre clases (Δmediana $\approx 0{,}036$), lo que sugiere que su discriminación depende más del ángulo de la columna que de la posición vertical de las articulaciones.

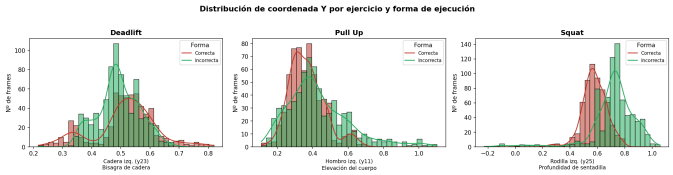

Figura 1. Distribución de la coordenada Y del landmark clave por ejercicio y forma de ejecución. El *squat* muestra la mayor separación entre distribuciones (Δmediana $= 0{,}154$), confirmando que la profundidad de la rodilla es altamente discriminante.

Cuadro II
ESTADÍSTICAS DEL LANDMARK CLAVE POR EJERCICIO Y FORMA

Ejercicio	Forma	Media	σ	Mediana
Deadlift (y23)	Correcta	0.492	0.071	0.490
	Incorrecta	0.515	0.107	0.526
Pull-up (y11)	Correcta	0.430	0.176	0.397
	Incorrecta	0.366	0.100	0.355
Squat (y25)	Correcta	0.720	0.165	0.732
	Incorrecta	0.579	0.088	0.578

IV-D2. Análisis de correlación entre articulaciones: Se calculó la correlación de Pearson entre las coordenadas Y de cuatro landmarks clave — hombro (11), cadera (23), rodilla (25) y tobillo (27) — para cada combinación de ejercicio y forma. La correlación cadera-rodilla resultó especialmente discriminante para el *squat*: la ejecución correcta presenta una correlación de $r = 0{,}841$ frente a $r = 0{,}688$ en la incorrecta ($\Delta = +0{,}153$). Esta diferencia tiene interpretación biomecánica directa: en un squat correcto, cadera y rodilla se flexionan y extienden de forma coordinada; en mala forma, esa sincronización se pierde. Para *deadlift* y *pull-up* el patrón se invierte: la ejecución incorrecta presenta mayor correlación ($\Delta = -0{,}127$ y $\Delta = -0{,}101$ respectivamente), lo que sugiere un movimiento más rígido y menos articulado en la mala forma.

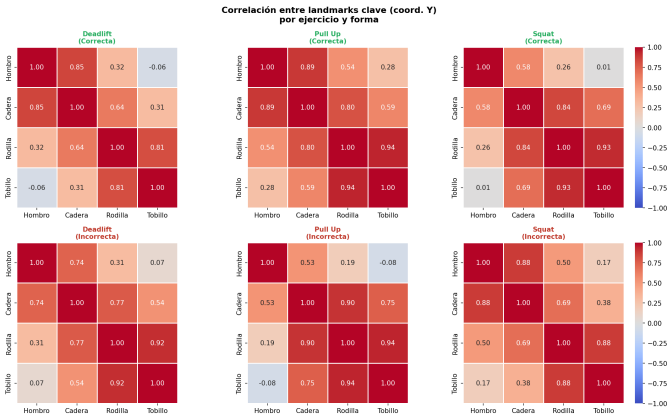

Figura 2. Matrices de correlación de Pearson entre landmarks clave (coordenada Y) por ejercicio y forma. La correlación cadera-rodilla en *squat* correcto ($r = 0{,}841$) es notablemente superior a la incorrecta ($r = 0{,}688$), reflejando la mayor sincronización articular en ejecuciones de calidad.

IV-D3. Proxies de ángulos articulares: Como aproximación previa al cálculo formal de ángulos articulares, se construyeron proxies basados en distancias verticales entre articulaciones: $|Y_{rodilla} - Y_{cadera}|$ para *squat*, $|Y_{codo} - Y_{hombro}|$ para *pull-up*, y $|Y_{hombro} - Y_{cadera}|$ para *deadlift*. Las medianas de los proxies articulares se muestran en la Tabla III.

El *squat* y el *pull-up* muestran diferencias sustanciales: la ejecución correcta alcanza mayor profundidad de rodilla y mayor extensión de brazo respectivamente. El *deadlift* no presenta diferencia apreciable en este proxy, confirmando que

Cuadro III
MEDIANAS DE LOS PROXIES ARTICULARES POR FORMA

Ejercicio	Correcta	Incorrecta	Δ
Squat (prof. rodilla)	0.132	0.089	+0.044
Pull-up (ext. brazo)	0.088	0.042	+0.045
Deadlift (incl. tronco)	0.181	0.185	−0.004

su discriminación requiere el ángulo real de columna que se calculará en la etapa de ingeniería de características. La Figura 3 ilustra la distribución completa de estos proxies.

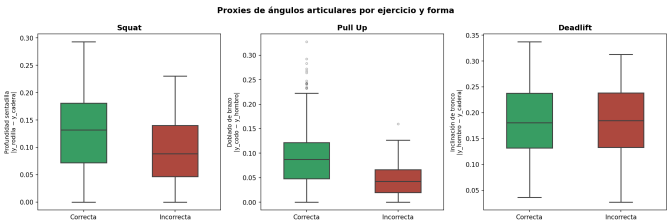

Figura 3. Boxplots de proxies de ángulos articulares por ejercicio y forma. *Squat* y *pull-up* muestran diferencias sustanciales. *Deadlift* presenta distribuciones solapadas, requiriendo el ángulo real de columna para su discriminación.

IV-E. Análisis de Calidad y Pipeline Alternativo para Squat Incorrecto

Durante la extracción inicial de landmarks se detectó que la tasa de detección de MediaPipe varía significativamente entre clases. La Tabla IV muestra los resultados sobre los 150 videos normalizados del conjunto de datos.

Cuadro IV
TASA DE DETECCIÓN DE LANDMARKS POR CLASE (PIPELINE ESTÁNDAR)

Ejercicio	Forma	Frames detectados	%
Deadlift	Correcta	747 / 750	99.6 %
Deadlift	Incorrecta	585 / 750	78.0 %
Pull-up	Correcta	666 / 750	88.8 %
Pull-up	Incorrecta	608 / 750	81.1 %
Squat	Correcta	732 / 750	97.6 %
Squat	Incorrecta	279 / 750	37.2 %

La clase *squat* incorrecta presentó una tasa de detección de apenas 37.2 %, con 16 de 25 videos con menos de 15 frames detectados. El análisis identificó dos causas:

1. **Selección de segmento sobre frames no detectables:** el algoritmo de normalización seleccionaba el segmento de mayor movimiento sobre la totalidad del video, incluyendo frames donde el sujeto no era visible o la pose era extrema. En videos de larga duración (>60 segundos) esto provocaba que el segmento extraído coincidiera con zonas fuera del rango de BlazePose.
2. **Degradación por compresión de video:** un intento de re-normalización con selección MediaPipe-first (escanear frames válidos antes de elegir el segmento) tampoco mejoró la tasa de detección, porque al escribir los frames seleccionados a un archivo `.mp4` con el codec `mp4v` y volver a leerlos, la compresión introducía artefactos suficientes para que BlazePose fallara en frames que eran detectables en el video original sin comprimir.

Dado que ambos intentos fallaron, se adoptó un **pipeline alternativo exclusivo para la clase *squat* incorrecto** que elimina la etapa de video normalizado como intermediario:

1. Escanear todos los frames del video **original** con MediaPipe y conservar los índices donde se detectó pose exitosamente, junto con sus landmarks ya extraídos.
2. Descartar videos con menos de $N_{min} = 15$ frames válidos.
3. Calcular la señal de movimiento exclusivamente sobre los frames válidos y encontrar el segmento de mayor actividad biomecánica.
4. Muestrear 30 índices del segmento y almacenar directamente los landmarks correspondientes en el dataset, sin reescribir ni releer ningún archivo de video.
5. Aumentar en **espacio de landmarks** mediante reflejo horizontal: $x_i \leftarrow 1 - x_i$ para cada landmark, con intercambio de pares izquierda↔derecha según la convención BlazePose.

Este enfoque garantiza detección del 100 % en los datos almacenados, ya que los landmarks se extraen en el mismo instante en que MediaPipe confirma la detección, sin pasos intermedios de compresión. Cabe señalar que la aumentación por ajuste de brillo, aplicada al resto de clases en espacio de píxeles, se omite aquí deliberadamente: modificar el brillo no altera las coordenadas de los landmarks y por tanto no aporta variabilidad al modelo. El reflejo horizontal sí constituye una aumentación semánticamente válida, pues refleja la ejecución del ejercicio en sentido contrario. Con los 10 videos originales y su versión en espejo se obtienen 600 filas en el dataset para esta clase (20 secuencias $\times$ 30 frames).

IV-F. Representación como Tensores

Los datos procesados se organizan en tensores de la forma [7]:

$$\mathbf{X} \in \mathbb{R}^{B \times T \times F} \tag{3}$$

donde B es el tamaño del lote (*batch*), $T = 30$ es el número de fotogramas y F es la dimensión de características por fotograma, compuesta por las coordenadas (x, y, z) de los 33 landmarks más los ángulos biomecánicos calculados.

Para evitar sesgos en el entrenamiento, se aplica balanceo de clases garantizando que cada lote contenga la misma proporción de ejecuciones correctas e incorrectas.

V. ARQUITECTURA DEL MODELO

Se propone una arquitectura híbrida CNN-LSTM que procesa la dinámica espacio-temporal del movimiento en dos etapas, como se ilustra a continuación.

V-A. Extracción Espacial mediante CNN

Se aplican capas convolucionales de una dimensión (Conv1D) sobre el vector de características de cada fotograma. Su función es detectar patrones locales de postura, como la alineación de la espalda o la profundidad del movimiento. Cada capa convolucional extrae un vector de características comprimido por fotograma.

V-B. Análisis Temporal mediante LSTM

Los vectores de características obtenidos por la CNN en cada fotograma se organizan en una secuencia temporal y se inyectan en una red LSTM. Las celdas de memoria y compuertas de la LSTM identifican la evolución del ejercicio a lo largo de los 30 fotogramas, detectando anomalías en la velocidad, profundidad o ritmo de ejecución.

V-C. Clasificación Final

Una capa densa completamente conectada con activación sigmoide genera una probabilidad $p \in [0, 1]$ de ejecución correcta, donde:

$$\hat{y} = \begin{cases} 1 & \text{si } p \geq \tau \quad \text{(ejecución correcta)} \\ 0 & \text{si } p < \tau \quad \text{(ejecución incorrecta)} \end{cases} \tag{4}$$

donde τ es el umbral de decisión, ajustado a $\tau = 0{,}30$ tras experimentación (véase §IX).

VI. Diseño del Sistema

Para gestionar la complejidad del pipeline y mantener separación de responsabilidades, el sistema se implementa mediante una arquitectura orientada a objetos con tres módulos principales:

- **DataManager:** transforma los videos crudos en flujos de datos estructurados. Encapsula la extracción de landmarks con MediaPipe, la normalización estructural y el cálculo de ángulos articulares.
- **DataLoader:** actúa como puente entre los datos procesados y la red neuronal. Organiza los datos en tensores, aplica el balanceo de clases y entrega lotes normalizados y equitativos durante el entrenamiento.
- **RecognitionModel:** define la arquitectura CNN-LSTM, gestiona el entrenamiento, el ajuste de hiperparámetros y la evaluación del rendimiento.

Esta separación permite modificar cualquier etapa del pipeline de forma independiente, facilitando la experimentación con diferentes configuraciones de arquitectura o estrategias de preprocesamiento.

VII. Preparación del Conjunto de Datos

Esta sección documenta las decisiones tomadas para transformar los datos crudos en tensores listos para el entrenamiento: manejo de valores faltantes, normalización, escalado y partición del conjunto.

VII-A. Manejo de Fotogramas Faltantes

MediaPipe BlazePose no detecta la pose en todos los fotogramas, especialmente cuando el sujeto está parcialmente fuera de cuadro o la iluminación es deficiente. Para el conjunto *squat_bad*, se registró una tasa de detección del 37.2 % en algunos videos (véase §IV). Se evaluaron dos estrategias:

- **Eliminación (drop):** descartar el fotograma fallido. Produce secuencias de longitud variable, incompatibles con la arquitectura tensorial fija.
- **Relleno hacia adelante (forward-fill):** propagar el último fotograma detectado al fotograma siguiente sin detección. Mantiene la longitud fija de 30 fotogramas y es semánticamente coherente, pues una postura estática es una aproximación válida de la postura real en un instante no detectado.

Se adoptó el **relleno hacia adelante**. Los videos con menos de 15 fotogramas detectados se descartan completamente por considerarse insuficientes para representar el movimiento.

VII-B. Escalado vs. Normalización

Estos conceptos se confunden frecuentemente pero operan sobre distintas propiedades estadísticas de los datos:

- **Normalización estructural** (aplicada en `DataManager`): traslada el esqueleto al origen de coordenadas (centro de cadera) y divide por la longitud del torso. Opera sobre la *geometría* del cuerpo para eliminar variaciones de posición de cámara y talla del sujeto. Es una normalización de dominio, no estadística.
- **Escalado estadístico** (aplicado en `DataLoader`): se aplica `StandardScaler` de *scikit-learn* sobre las 108 características del vector de entrada. Resta la media y divide por la desviación estándar de cada característica, resultando en distribuciones con $\mu = 0$ y $\sigma = 1$. Esto evita que características con mayor varianza dominen el gradiente durante el entrenamiento.

Ambas transformaciones son complementarias y necesarias. El escalado estadístico se ajusta *exclusivamente* sobre el conjunto de entrenamiento y luego se aplica a validación y prueba para evitar fuga de información (*data leakage*).

VII-C. Partición del Conjunto de Datos

La partición se realiza a nivel de video, no de fotograma, para evitar que fotogramas del mismo video aparezcan en conjuntos distintos (leakage temporal). Se aplica un split estratificado por la combinación `(ejercicio, etiqueta)` para garantizar representación balanceada de las 6 clases en cada subconjunto.

Los videos de aumentación por reflejo horizontal (generados para *squat_bad*) se asignan exclusivamente al conjunto de entrenamiento, evitando que versiones modificadas de videos de prueba contaminen la evaluación.

Cuadro V
DISTRIBUCIÓN DEL CONJUNTO DE DATOS

Subconjunto	Videos	Correctos	Incorrectos
Entrenamiento (70 %)	93	50	43
Validación (15 %)	19	12	7
Prueba (15 %)	19	12	7
Total	131	74	57

VIII. IMPLEMENTACIÓN Y CONFIGURACIÓN DEL MODELO

VIII-A. Hiperparámetros

La Tabla VI resume los hiperparámetros del modelo final, determinados mediante experimentación sistemática documentada en §IX.

Cuadro VI
HIPERPARÁMETROS DEL MODELO FINAL

Parámetro	Valor	Justificación
`conv_filters`	64	Balance capacidad/overfitting
`lstm_units`	128	Suficiente memoria temporal
`lstm_layers`	2	Jerarquía temporal de 2 niveles
`dropout`	0.3	Regularización moderada
`lr`	10^{-3}	Valor estándar para Adam
`batch_size`	16	Ajustado al tamaño del dataset
`epochs`	50	Con early stopping (patience=10)
`umbral`	0.30	Optimizado en validación
`seed`	42	Reproducibilidad

VIII-B. Detalles de Implementación del CNN

A diferencia del uso convencional de `nn.Conv1d` de PyTorch, la capa convolucional se implementó manualmente mediante `nn.Parameter` y operaciones tensoriales básicas, con el objetivo de comprender el mecanismo interno de la convolución. El procedimiento sigue cuatro pasos:

1. **Padding:** se agregan ceros al inicio y al final de la secuencia temporal para mantener la longitud de salida igual a la de entrada.
2. **Ventanas deslizantes:** mediante `tensor.unfold()` se extraen todas las ventanas de $k = 3$ fotogramas consecutivos.
3. **Producto punto:** cada ventana se multiplica matricialmente por la matriz de filtros aplanada.
4. **Suma del sesgo:** se agrega el vector de bias y se reorganizan las dimensiones de salida.

Los pesos de los filtros se inicializan con distribución normal escalada ($\times 0{,}01$) y los biases en cero.

IX. EXPERIMENTOS Y RESULTADOS

IX-A. Protocolo de Entrenamiento

El modelo se entrena con Binary Cross-Entropy Loss (BCELoss) y el optimizador Adam. Se implementó *early stopping*: si la pérdida de validación no mejora durante `patience` épocas consecutivas, el entrenamiento se detiene y se restauran los mejores pesos observados. Para garantizar reproducibilidad, se fija la semilla aleatoria en 42 para Python, NumPy y PyTorch.

IX-B. Experimentos de Ajuste

Se realizaron cuatro experimentos documentados en la Tabla VII.

Cuadro VII
COMPARACIÓN DE EXPERIMENTOS (TEST SET, 19 VIDEOS)

Exp.	Descripción	Umbral	Acc.	F1	Recall
1	Baseline (sin semilla)	0.50	73.68 %	0.815	0.917
2	Exp.1 + umbral reducido	0.40	78.95 %	0.857	1.000
3	dropout=0.4, lr=5×10^{-4}	0.50	63.16 %	0.696	0.667
4	**seed=42, umbral=0.30**	**0.30**	**73.68 %**	**0.815**	**0.917**

El Experimento 3 demostró que aumentar el dropout a 0.4 y reducir el learning rate a 5×10^{-4} produce underfitting: el modelo deja de aprender los patrones de la clase correcta con suficiente confianza (Recall cae a 0.667) sin compensar con una mejora significativa en la detección de forma incorrecta.

IX-C. Resultados del Modelo Final

El modelo del Experimento 4 (seed=42, umbral=0.30) constituye el resultado reproducible final. La Tabla VIII presenta las métricas sobre el conjunto de prueba.

Cuadro VIII
MÉTRICAS FINALES — EXPERIMENTO 4 (TEST SET)

Métrica	Valor
Accuracy	73.68 %
F1-score	0.8148
Precision	0.7333
Recall	0.9167

IX-D. Matriz de Confusión

Cuadro IX
MATRIZ DE CONFUSIÓN — EXPERIMENTO 4

	Pred. Incorrecto	Pred. Correcto
Real Incorrecto	3	4
Real Correcto	1	11

El modelo clasifica correctamente 11 de 12 ejecuciones correctas (Recall=0.917) y 3 de 7 ejecuciones incorrectas (especificidad=0.429). El sesgo hacia la clase positiva es consistente con el desbalance leve del conjunto de entrenamiento (50 correctos vs. 43 incorrectos) y con el tamaño reducido del dataset.

IX-E. Limitaciones

La principal limitación del sistema es el tamaño del conjunto de datos: 131 videos en total, con apenas 19 videos en el conjunto de prueba. Con un test set de este tamaño, cada clasificación errónea representa aproximadamente 5.3 puntos porcentuales de accuracy, lo que introduce alta varianza en las métricas reportadas. Esta limitación es inherente al dataset disponible y motiva como trabajo futuro la recolección de más datos de ejecución incorrecta, que es la clase más difícil de distinguir para el modelo.

X. Conclusión y Trabajo Futuro

Este trabajo presenta el diseño e implementación de un sistema de reconocimiento automático de la calidad de ejecución biomecánica de ejercicios de fuerza, fundamentado en una arquitectura híbrida CNN-LSTM y en la extracción de representaciones esqueletales mediante MediaPipe BlazePose.

El sistema fue implementado completamente en Python con PyTorch, siguiendo una arquitectura orientada a objetos con tres módulos: `DataManager`, `DataLoader` y `RecognitionModel`. La capa convolucional se implementó a nivel de operaciones tensoriales básicas (`nn.Parameter` + `unfold` + producto matricial), prescindiendo de la abstracción de alto nivel de PyTorch con fines pedagógicos.

El modelo final (Experimento 4) alcanzó una *accuracy* de 73.68 %, un F1-score de 0.815 y un Recall de 0.917 sobre el conjunto de prueba, clasificando correctamente 11 de 12 ejecuciones correctas y 3 de 7 incorrectas. La principal limitación es el tamaño reducido del dataset (131 videos), que introduce alta varianza en las métricas e impide una evaluación estadísticamente robusta.

Como trabajo futuro se identifican tres líneas prioritarias: (1) ampliar el dataset con más videos de forma incorrecta, clase que el modelo detecta con menor confianza (especificidad = 0.429); (2) extender la evaluación por ejercicio individual para identificar en cuáles el modelo generaliza mejor; y (3) explorar arquitecturas con atención temporal (*Transformer*) que puedan capturar dependencias de largo plazo en la secuencia de movimiento sin requerir la parametrización manual del LSTM.

Reconocimientos

Los autores agradecen al personal docente del curso de Inteligencia Artificial, Ciclo IV, de la Universidad de Costa Rica por la orientación brindada durante el desarrollo de este proyecto.

Referencias

[1] P. Ren, C. Xiao, X. Chang, P.-Y. Huang, Z. Li, B. B. Gupta, others, and X. Wang, "A survey on 3D skeleton-based action recognition using learning method," *IEEE Access*, vol. 8, pp. 223 409–223 427, 2020.

[2] Google. (2024) Pose landmarker | MediaPipe | google for developers. Accessed: Apr. 2025. [Online]. Available: https://ai.google.dev/edge/mediapipe/solutions/vision/pose_landmarker

[3] A. Ullah, J. Ahmad, K. Muhammad, M. Sajjad, and S. W. Baik, "Action recognition in video sequences using deep bi-directional LSTM with CNN features," *IEEE Access*, vol. 6, pp. 1155–1166, 2018.

[4] D. A. Winter, *Biomechanics and Motor Control of Human Movement*, 4th ed. Hoboken, NJ, USA: Wiley, 2009.

[5] Y. Zhou, C. Barnes, J. Lu, J. Yang, and H. Li, "On the continuity of rotation representations in neural networks," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, Long Beach, CA, USA, 2019, pp. 5745–5753.

[6] H. Abdillah. (2023) Workout/fitness video dataset. Accessed: Apr. 2025. [Online]. Available: https://www.kaggle.com/datasets/hasyimabdillah/workoutfitness-video/data

[7] W3Schools. (2024) What is a tensor? Accessed: Apr. 2025. [Online]. Available: https://www.w3schools.com/ai/ai_tensors.asp